

Basics of Java and Git

Friday, October 2, 2026 5:17 PM

1. How an application works

User: the person interacting with the application.

Frontend: the screens and controls the user sees and uses.

Backend: code that processes requests and returns responses.

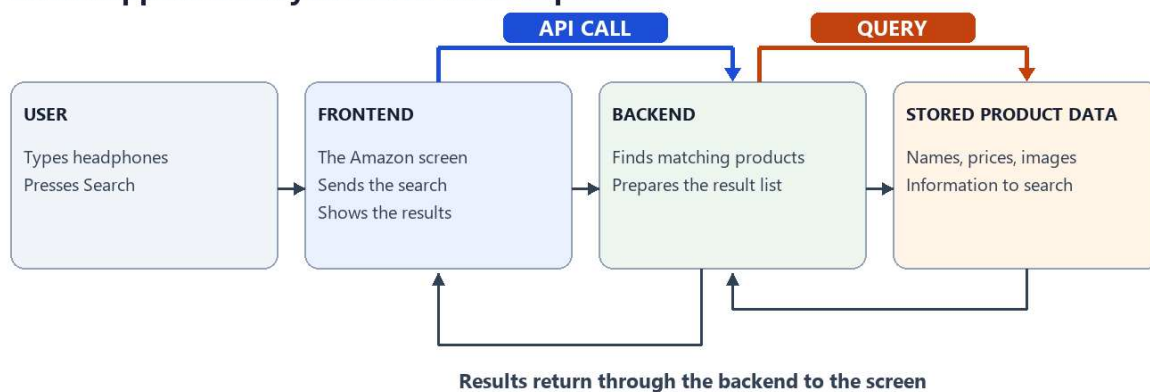
Database: organised storage for application data.

API call: a request between application components. In a product search, the frontend asks the backend for matching products.

Query: a request to retrieve or change stored data.

Amazon search example: user searches for headphones → frontend sends an API request → backend looks up product data → response returns → frontend displays results.

What happens when you search for headphones?



API call = the screen asking the backend for matching products.

Query = the backend asking stored data to find matches.

This is a simplified teaching example, not a description of Amazon's internal design.

This is a simplified example; real product search may use search services and cached data.

2. Development tools

JDK: Java Development Kit; includes the compiler, runtime and development tools. Java 25 is an LTS (long-term support) release.

IDE: an environment for writing, running and debugging code. Examples include IntelliJ IDEA and VS Code with Java extensions.

Git: a version-control tool that records changes to code.

3. Java and backend applications

Java classes model application data and behaviour. Backend applications use this code to process requests and implement business rules.

Core foundations include classes, objects, constructors, methods, static members, references, Java execution, packages and OOP.

4. The main method

```
public static void main(String[] args) {
    System.out.println("Application started");
}
```

main: the entry point used in our conventional Java programs.

public: makes the method accessible outside its class.

static: allows the method to be called without creating an object of its class.

void: the method returns no value.

String[] args: command-line arguments passed to the program.

Java 25 also supports other entry-point forms.

5. Classes, objects, constructors and methods

Class: defines the fields and behaviour of its objects.

Object: an instance of a class, with its own instance-field values.

Field: a variable declared in a class to hold data.

Constructor: initialises an object during creation; has the class name and no return type.

Method: a named block of code that performs an operation.

this: a reference to the current object.

```
class Product {
    String name;
    int priceRupees;

    Product(String name, int priceRupees) {
        this.name = name;
        this.priceRupees = priceRupees;
    }

    void showProduct() {
        System.out.println(name + " " + priceRupees);
    }

    void changePrice(int newPrice) {
        this.priceRupees = newPrice;
    }
}
```

In this.name = name, this.name is the object's field; name is the constructor parameter.

new Product("Boat", 1500) creates a Product object and invokes its constructor.

showProduct() is an instance method that prints details, rather than a getter that returns a value.

Java provides a default no-argument constructor only when the class declares no constructor.

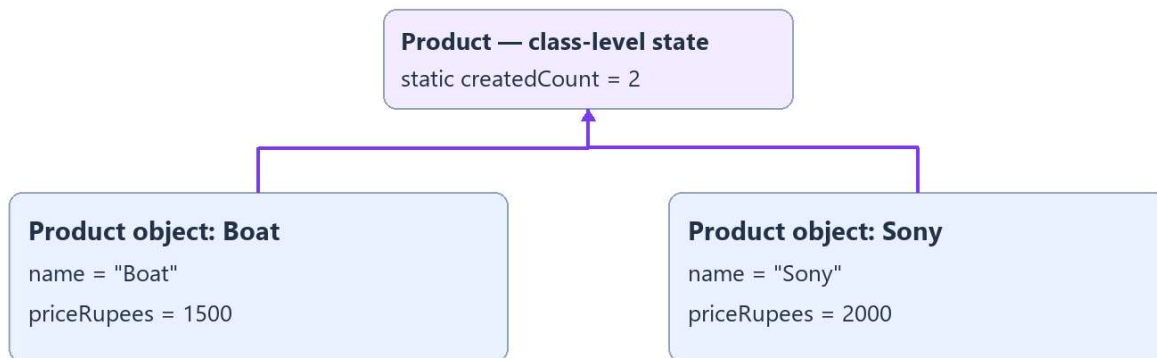
6. Static members

A static member belongs to the class. An instance field belongs to each individual object.

Product.createdCount can track how many Product objects have been created: a static int

begins at 0 and increases when the constructor increments it.
After two objects are created, the shared counter is 2; each object still has its own name and price.
static does not mean constant. Access static members using the class name.

One class, separate objects, one shared counter



Each object has its own fields. `Product.createdCount` belongs to the class.

7. Primitive and reference variables

Primitive variables hold primitive values, such as 2. Reference variables hold references to objects, or null.

```
Product headphone1 = new Product("Boat", 1500);
Product alias = headphone1;
alias.changePrice(1800);
headphone1.showProduct(); // Boat 1800
```

`alias = headphone1` copies a reference, not the object. Both variables refer to the same `Product`.

Changing the shared object is visible through both references. Reassigning one variable changes only which object that variable refers to.

8. Java execution and memory

JDK: tools and runtime for developing Java programs.

JRE: runtime environment, including the JVM and supporting libraries.

JVM: Java Virtual Machine; loads and executes Java bytecode.

Execution: `.java` source → `javac` compiler → `.class` bytecode → JVM → output.



Stack: method-call frames contain local variables, including primitive values and object references.

Heap: stores objects and arrays. Object fields may hold primitive values or references.

Product.priceRupees is a primitive field inside a heap object. Product.name is a reference field pointing to a String object.

Stack and heap: where your variables and objects are

A snapshot inside main() after these four statements have run

```
int quantity = 2;
Product headphone1 = new Product("Boat", 1500);
Product headphone2 = new Product("Sony", 2000);
Product alias = headphone1;
```

STACK

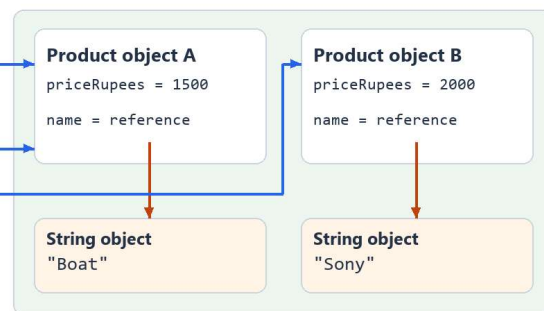
Local variables in the main() stack frame

Variable	Type	Stored value
quantity	int	2
headphone1	Product	Ref A
alias	Product	Ref A
headphone2	Product	Ref B

headphone1 and alias refer to the same object.

HEAP

Objects and their fields



Stack: local primitive values AND local references. **Heap:** objects, arrays and their fields.

References can also be fields inside heap objects, such as Product.name. Ref A / B are labels, not addresses.

Simplified teaching view. Static fields and other JVM memory areas are omitted.

A method's stack frame is removed when the method completes. Unreachable objects are eligible for garbage collection.

This is a simplified logical view; static state and other JVM memory areas are omitted. Ref A and Ref B are labels, not literal memory addresses.

9. WORA: Write Once, Run Anywhere

Java bytecode can run on different operating systems with a compatible Java runtime, without rewriting the application for each system.

The JVM is platform-specific; the bytecode is portable. Running compiled code needs a compatible runtime, rather than necessarily a full development kit.

Runtime versions, required libraries and platform-specific dependencies affect compatibility.

10. Packages

A package groups related types and provides a namespace, helping organise code and avoid naming conflicts.

```
package model;           // Product belongs to model
import model.Product;    // Used by another source file
```

model.Product is the fully qualified class name. Package names conventionally use lowercase

letters.

Classes in the same package do not need imports for one another. An import allows a short class name; it does not bypass access restrictions.

public declarations can be accessed from other packages, subject to their enclosing type's accessibility. No access modifier means package access.

Using Product from another package requires an accessible class, constructor and any methods being called.

11. Object-Oriented Programming

OOP organises programs around objects that combine data and behaviour. Product combines name and priceRupees with methods such as showProduct() and changePrice().

Encapsulation: controls access to data and protects rules.

Inheritance: creates a specialised type from an existing type.

Polymorphism: allows objects with different behaviour to be used through a common type.

Abstraction: exposes essential operations while hiding unnecessary details.

12. Inheritance

Inheritance creates a specialised class from an existing class. Product is the parent (superclass); DiscountedProduct is the child (subclass).

extends establishes the parent-child relationship. DiscountedProduct is a Product with additional discount data and specialised price calculation.

super(name, priceRupees) invokes the parent constructor to initialise the parent-managed state. Constructors are not inherited.

The child inherits accessible methods such as showProduct() and changePrice(). Private parent fields cannot be accessed directly by child-class code; access is provided through methods.

13. Encapsulation

Encapsulation combines data with behaviour and controls access to that data.

private fields prevent unrestricted direct access. Public methods provide controlled reads and updates.

getPriceRupees() returns the stored price. changePrice() rejects negative prices before updating the field.

Encapsulation protects rules; it does not require a setter for every field.

14. Method overriding

Overriding supplies a specialised implementation of an inherited instance method with the same signature and a compatible return type.

Product.getFinalPrice() returns the regular price. DiscountedProduct.getFinalPrice() returns the price after discount, with a minimum of zero.

@Override lets the compiler check that the method correctly overrides an inherited method.

The inherited showProduct() calls getFinalPrice(); that call uses the overridden child implementation for a DiscountedProduct object.

15. Polymorphism

Polymorphism allows a parent-type reference to refer to a child object.

The reference type determines which operations are available at compile time. The actual object's class determines which overridden instance-method implementation executes at

runtime.

Product discounted = new DiscountedProduct("Sony", 2000, 300) uses a Product reference for a DiscountedProduct object.

displayProduct(Product product) accepts both regular and discounted products. The same call produces behaviour appropriate to each object.

16. Abstraction

Abstraction exposes useful operations while hiding unnecessary implementation details.

getFinalPrice() provides the payable price without requiring the caller to perform the discount calculation.

Encapsulation controls access to state; abstraction presents a useful view of an operation.

Abstraction does not always require the abstract keyword or an interface. Interfaces and abstract classes are language mechanisms for expressing abstraction.

Product example

```
class Product {
    private String name;
    private int priceRupees;
    static int createdCount;

    Product(String name, int priceRupees) {
        this.name = name;
        this.priceRupees = priceRupees;
        createdCount++;
    }

    public int getPriceRupees() {
        return priceRupees;
    }

    public int getFinalPrice() {
        return priceRupees;
    }

    public void changePrice(int newPrice) {
        if (newPrice < 0) {
            System.out.println("Price cannot be negative");
            return;
        }
        priceRupees = newPrice;
    }

    public void showProduct() {
        System.out.println(name + " | Final price: " + getFinalPrice());
    }
}

class DiscountedProduct extends Product {
    private int discountRupees;
```

```

        DiscountedProduct(String name, int priceRupees, int discountRupees
    ) {
        super(name, priceRupees);
        this.discountRupees = discountRupees;
    }

    @Override
    public int getFinalPrice() {
        return Math.max(0, getPriceRupees() - discountRupees);
    }
}

public class Demo {
    static void displayProduct(Product product) {
        product.showProduct();
    }

    public static void main(String[] args) {
        Product regular = new Product("Boat", 1500);
        Product discounted = new DiscountedProduct("Sony", 2000, 300);
        displayProduct(regular);
        displayProduct(discounted);
        discounted.changePrice(2500);
        displayProduct(discounted);
        System.out.println(Product.createdCount);
    }
}

```

Expected output:

Boat | Final price: 1500

Sony | Final price: 1700

Sony | Final price: 2200

2

This example uses non-negative starting prices and discounts. Price updates are validated; constructor and discount validation can extend the model.

17. Git, GitHub and version control

Version control records changes over time, so we can inspect earlier versions, understand changes and work on separate features.

Git is a distributed version-control tool. A local repository holds project history, allowing local commits without an internet connection.

GitHub hosts remote Git repositories and supports collaboration, code review and pull requests.

Saving a file changes working files. Committing records selected changes locally. Pushing shares commits with a remote repository.

Working files, staging, commits and branches

Working tree: the project files currently checked out on your computer.

Staging area (index): the content selected for the next commit. `git add` stages the current content; later edits must be staged again.

Repository: Git's stored history and metadata, normally in the `.git` directory locally.

Commit: a project snapshot with an identifier, author, message and links to earlier commits.

Branch: a movable name pointing to a commit, providing a line of development. `main` is a conventional branch name.

HEAD: indicates the currently checked-out branch in normal branch-based work.

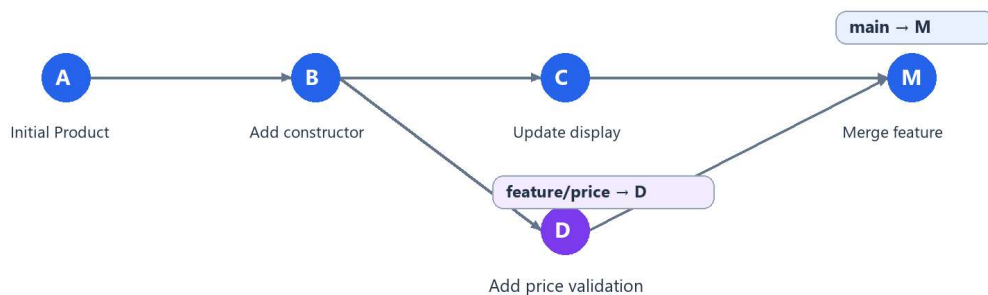
Remote: a named connection to another repository. `origin` is a conventional remote name.

Git: save versions locally, work on branches, share on GitHub



Versions and branches inside the LOCAL repository

A commit is a saved project snapshot. A branch is a movable name pointing to a commit.



Branches share earlier commits. Switching branches updates tracked files to that branch's version.

Commit saves locally. Push shares commits. Neither action happens automatically when you save a file.

A-M are teaching labels, not real commit IDs. The graph shows history after a merge.

1. Verify Git and configure commit identity

```
git --version
```

```
git config --global user.name "Your Name"
```

```
git config --global user.email "YOUR_COMMIT_EMAIL"
```

The name and email identify commit authors; they do not sign you into GitHub.

`YOUR_COMMIT_EMAIL` can be your GitHub-provided no-reply address.

2. Start tracking the Java project

Run commands inside the project folder containing the Java source files.

```
git init -b main
```

```
git status
```

`git init` creates a local repository. `git status` shows the current branch and staged, modified or untracked files.

3. Ignore generated files

A `.gitignore` file lists files Git should leave untracked. A simple Java project can use:

```
*.class
```

```
out/  
build/  
.idea/  
*.iml
```

Source files remain tracked. Ignore rules do not remove files already tracked by Git.

4. Stage and record the first version

```
git add .  
git status  
git diff --staged  
git commit -m "Add Product class and demo"  
git log --oneline
```

git add . stages changes under the current directory. git diff --staged shows staged differences. git commit records the staged snapshot locally. git log --oneline lists abbreviated commit IDs and messages.

5. Record another version

Example change: introduce a changePrice() method in Product.java.

```
git diff  
git add .  
git diff --staged  
git commit -m "Add product price updates"  
git log --oneline
```

git diff shows unstaged differences in tracked files. Each commit records a meaningful checkpoint; it does not require a separate project folder.

6. Connect to GitHub and push

Create an empty GitHub repository named java-oop-demo, without an initial README, licence or .gitignore, for this existing-project example. Replace YOUR_USERNAME in the URL with your username.

```
git remote add origin https://github.com/YOUR\_USERNAME/java-oop-demo.git  
git remote -v  
git push -u origin main
```

git remote add names the remote connection; it does not upload code. git push uploads commits and updates the remote branch. -u records the upstream relationship so later pushes can use git push.

GitHub HTTPS authentication uses supported credentials such as Git Credential Manager or a personal access token, rather than your account password.

7. Create and use a feature branch

```
git switch -c feature/price-validation  
git branch
```

git switch -c creates a branch and switches to it. git branch lists local branches; an asterisk marks the current one.

Example change: reject a negative price inside changePrice().

```
git add .  
git commit -m "Reject negative product prices"  
git switch main
```

With all changes committed, switching to main shows main's tracked files without the feature commit. Switching back shows the feature version:

```
git switch feature/price-validation
```

Branches share earlier history. A branch is not a separate copy of the project folder.

8. Merge the feature into main

```
git switch main
```

```
git merge feature/price-validation
```

```
git log --oneline --graph --all --decorate
```

```
git push
```

git merge integrates the feature history into the current branch. In this simple demo, main has not advanced, so Git normally performs a fast-forward rather than creating a new merge commit.

The diagram shows the other case: both branches advanced, and a merge commit joins their histories. Overlapping edits may require resolving a merge conflict.

A GitHub pull request proposes merging changes for review; it is a GitHub workflow, not the git pull command.

Other useful commands

```
git clone https://github.com/YOUR\_USERNAME/java-oop-demo.git
```

```
git fetch origin
```

```
git pull --ff-only
```

clone creates a local copy of a repository and its history. Use it as an alternative to init for an existing remote project, in a separate folder.

fetch downloads remote history and updates remote-tracking references; it does not integrate changes into the current branch.

pull fetches and integrates changes. --ff-only permits integration only when it can advance the branch without creating a merge commit; it stops if histories have diverged.

Everyday workflow

Edit → inspect changes → stage → inspect staged changes → commit → push.

A commit records staged content, not every unsaved or unstaged change. A push shares commits, not arbitrary uncommitted files.

References

[Official Git tutorial](#)

[Git branch switching](#)

[GitHub remote repositories](#)